C++ Boolean (bool)

The Language of Logic & Decision Making

1 / 0 | TRUE / FALSE

Fundamental Programming Series

1. Introduction to Booleans

In computer science, everything eventually boils down to two states: **On** or **Off**. In C++, the `bool` data type is the tool we use to represent these two states.

A Boolean variable can store only one of two possible values:

- `true` (Logical Yes)
- `false` (Logical No)

> **Why it matters:** Booleans are the "brain" of your program. Without them, your code could not make decisions, check permissions, or know when to stop a task.

Creating Boolean Variables

```
bool isCodingFun = true;
bool isRaining = false;
```

2. Boolean Output & Numbers

Computers internally treat `true` as **1** and `false` as **0**. When you print a boolean value using `cout`, C++ follows this numeric representation by default.

Example Output:

```
bool result = true;
cout << result;  // Output: 1

bool failure = false;
cout << failure; // Output: 0
```

Fun Fact: In C++, technically any non-zero number is treated as `true`, but the `bool` keyword specifically ensures we only deal with 1 and 0.

Outputting as Text

If you prefer to see "true" or "false" in the console, you can use the `boolalpha` manipulator:

```
cout << boolalpha << result; // Output: true
```

3. Booleans from Comparisons

We rarely assign `true` or `false` manually. Instead, we ask questions about data. The answers to those questions are always Booleans.

```
int x = 10;
int y = 5;

bool check = (x > y);  // Is 10 greater than 5? → true
cout << (x == y);      // Is 10 equal to 5? → false (0)
```

Common Comparison Operators:

Operator	Meaning
==	Equal To
!=	Not Equal To
>	Greater Than
<	Less Than
>=	Greater or Equal

4. Logical Operators

Logical operators allow you to combine multiple boolean questions into a single complex decision.

The "Big Three" Operators:

1. AND (&&): True only if BOTH are true.

```
(10 > 5 && 5 > 2) // true
```

2. OR (||): True if at least ONE is true.

```
(10 < 5 || 5 > 2) // true
```

3. NOT (!): Reverses the value.

```
!(10 > 5) // false
```

```
bool hasKey = true;
bool hasGold = false;

bool canOpenDoor = (hasKey || hasGold); // true
```

5. Booleans in Decision Making

The most frequent use of a boolean is inside an `if` statement. The block of code following the `if` only runs if the boolean value is `true`.

```
bool isLoggedIn = true;

if (isLoggedIn) {
  cout << "Welcome to your Dashboard!";
} else {
  cout << "Access Denied. Please login.";
}
```

The Logic "Flow"

Because the condition inside the parenthesis `(...)` is expected to be a boolean, you don't need to write `if (isLoggedIn == true)`. Writing `if (isLoggedIn)` is cleaner and preferred by professional developers.

6. Booleans in Loops and Functions

Controlling Loops

Booleans can act as a "toggle" to start or stop a loop.

```
bool keepRunning = true;
while (keepRunning) {
    // ... logic ...
    keepRunning = false; // The loop stops after one turn
}
```

Returning from Functions

Functions that check for a condition (often called "predicate functions") return a boolean result.

```
bool isEven(int num) {
  return (num % 2 == 0);
}

// Usage:
if (isEven(10)) { cout << "10 is even"; }
```

7. Best Practices & Challenges

1. Naming Conventions: Always prefix boolean variables with words like **is**, **has**, or **can**.
Bad: `data`
Good: `hasData`, `isValid`

2. Keep Logic Simple: Avoid "Double Negatives."
Confusing: `!isNotLoggedIn`
Clear: `isLoggedIn`

Code Challenge Lab

1. **The Age Check:** Create a program that takes an age as input and stores `true` in a boolean variable `canVote` if the age is 18 or older.
2. **Logic Master:** Try to predict the result:
   ```
   bool x = (10 > 5) && !(5 == 10) || (2 > 10);
   ```

End of Module: C++ Booleans
Mastered Logical Decision Making